

Nathan Krieger
04/08/26
HW4

Q1. Answer the questions based on the following Graph:

$N = \{1, 2, 3, 4, 5, 6\}$

$N_0 = \{1\}$

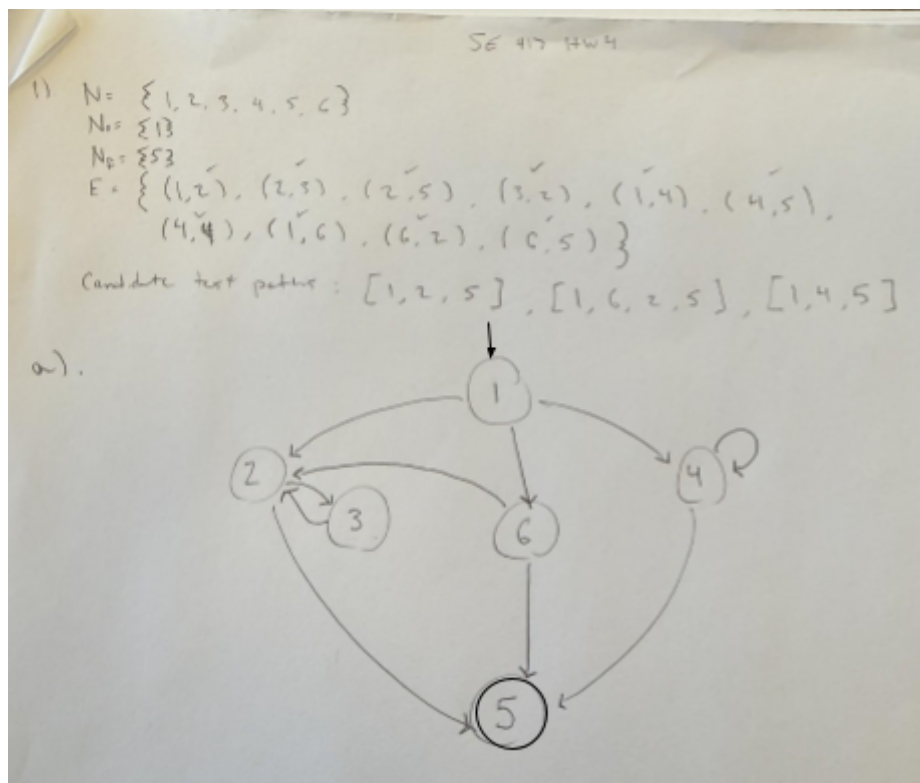
$N_f = \{5\}$

$E = \{ \{1, 2\}, \{2, 3\}, \{2, 5\}, \{3, 2\}, \{1, 4\}, \{4, 5\}, \{4, 4\}, \{1, 6\}, \{6, 2\}, \{6, 5\} \}$

And the following candidate test paths:

$[1, 2, 5], [1, 6, 2, 5], [1, 4, 5]$

(a) Draw the graph (you can use the tool from the book website) and include a screenshot/picture (10)



(b) How many simple paths are there (leave out length 0- single nodes)? (5)

1,2
1,6
1,4
2,3

2,5

3,2

4,4

4,5

6,5

6,2

1,2,3

1,2,5

1,4,5

1,6,5

1,6,2

2,3,2

3,2,3

3,2,5

6,2,5

6,2,3

1,6,2,3

1,6,2,5

Total = 22

(c) List just the length one paths. Include the exclamation points and star on paths you would add if you were creating this by hand.

‘*’ means the first node is the same as the last

‘!’ means the path cannot be extended because the last node has no outgoing edges

[1, 2]

[2, 3]

[2, 5]!

[3, 2]

[1, 4]

[4, 5]!

[4, 4]*

[1, 6]

[6, 2]

[6, 5]!

(d) List the prime paths (10)

1 6 2 5

1 6 2 3

1 2 3
 1 2 5
 1 6 5
 1 4 5
 4 4
 3 2 3
 2 3 2
 3 2 5

(e) Does the given set of test paths satisfy (1) Edge Coverage (2) Edge Pair Coverage (3) Node coverage? If not add test paths to complete those requirements (do not delete existing test paths) (10)

Test set: [1,2,5], [1,6,2,5], [1,4,5]

Edge Coverage: No. Several edges are missing: (2,3), (3,2), (4,4), (6,5)

Added tests:

[1, 6, 5] - covers edge (6,5)
 [1, 2, 3, 2, 5] - covers edge (2,3) and (3,2)
 [1, 4, 4, 5] - covers edge (4,4)

Edge Pair Coverage: No. Several edge pairs are missing:

1,2,3
 2,3,2
 3,2,3
 3,2,5
 6,2,3
 4,4,4

Added tests:

[1,2,3,2,5]
 [1,6,2,3,2,3,2,5]
 [1,4,4,4,5]

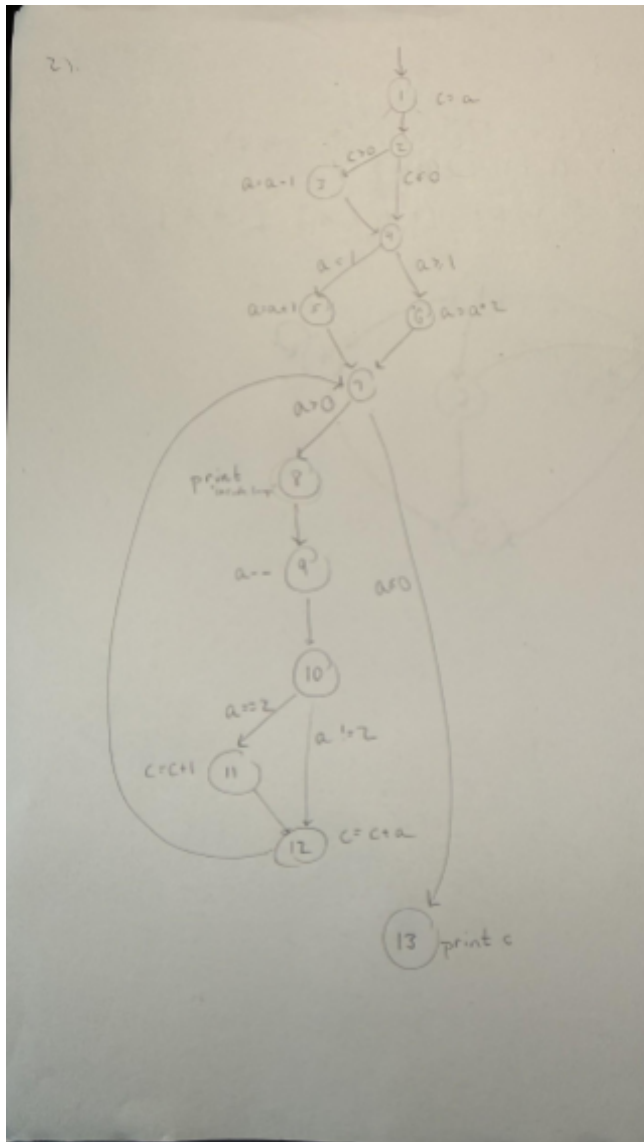
Node Coverage: No. The tests do not cover node 3.

Added tests:

[1,2,3,2,5]

Q2. Draw a control flow graph for the following program fragment. Use the rules from our book (i.e. include extra nodes for loops) (15)

[code]



Q3. Using a mutation testing tool

3.

=====

=====

- Timings

=====

=====

> pre-scan for mutations : < 1 second

```

> scan classpath : < 1 second
> coverage and dependency analysis : < 1 second
> build mutation tests : < 1 second
> run mutation analysis : 2 seconds
-----
> Total : 2 seconds
-----
> 0 tests took longer than 2000 ms
> Slowest test (test1(com.example.TriangleTest)) took 10 ms
> Largest test (test1(com.example.TriangleTest)) covered 23 blocks
=====
=====
- Statistics
=====
=====
>> Line Coverage (for mutated classes only): 18/23 (78%)
>> 3 tests examined
>> Generated 61 mutations Killed 35 (57%)
>> Mutations with no coverage 9. Test strength 67%
>> Ran 95 tests (1.56 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 46.501 s
[INFO] Finished at: 2026-04-08T18:03:47-05:00
[INFO] -----

```

4.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	78% 18/23	57% 35/61	67% 35/52

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.example	1	78% 18/23	57% 35/61	67% 35/52

Report generated by [PIT](#) 1.23.0

Enhanced functionality available at [arcmutate.com](#)

5.

15	1. removed conditional - replaced equality check with true → KILLED
	2. removed conditional - replaced equality check with false → KILLED
16	1. Replaced integer addition with subtraction → KILLED
18	1. removed conditional - replaced equality check with true → KILLED
	2. removed conditional - replaced equality check with false → KILLED
19	1. Replaced integer addition with subtraction → KILLED
21	1. removed conditional - replaced equality check with true → KILLED
	2. removed conditional - replaced equality check with false → KILLED
	1. removed conditional - replaced comparison check with true → SURVIVED Covering tests
	2. Replaced integer addition with subtraction → KILLED
	3. removed conditional - replaced comparison check with false → SURVIVED Covering tests
	4. changed conditional boundary → SURVIVED Covering tests
	5. removed conditional - replaced comparison check with true → KILLED
22	6. removed conditional - replaced comparison check with false → KILLED
	7. changed conditional boundary → SURVIVED Covering tests
	8. changed conditional boundary → SURVIVED Covering tests
	9. Replaced integer addition with subtraction → KILLED
	10. removed conditional - replaced comparison check with false → KILLED
	11. removed conditional - replaced comparison check with true → SURVIVED Covering tests
	12. Replaced integer addition with subtraction → KILLED
23	1. replaced return value with null for com/example/Triangle::classify → NO_COVERAGE
25	1. replaced return value with null for com/example/Triangle::classify → KILLED
26	1. removed conditional - replaced comparison check with true → KILLED
28	2. changed conditional boundary → SURVIVED Covering tests

6.

21	1. removed conditional - replaced equality check with true → KILLED
	2. removed conditional - replaced equality check with false → KILLED
	1. removed conditional - replaced comparison check with true → SURVIVED Covering tests
	Covered by tests:
	• com.example.TriangleTest.test1(com.example.TriangleTest)

This test changes line 21 from:

```
if (trian == 0) {
```

To:

```
if (true) {
```

This means that the inner logic will always be hit. To evaluate why this mutation survived, we can look at the test that covers it which is test1:

```
@Test
public void test1() {
    final TriangleType type = Triangle.classify(4, 5, 6);
    assertEquals(SCALENE, type);
}
```

Since none of these sides are equal, by the time we get to line 21, trian is 0, so this would have evaluated true regardless of the mutation.

There are tests for isosceles and equilateral triangles but, they do not verify that the condition `trian == 0` is working correctly. The test suite does not ensure that this block is skipped when `trian != 0`, so the incorrect behavior introduced by the mutation is not detected.

To kill this mutation, additional tests would be needed that specifically verify correct behavior when `trian != 0` and ensure the program does not incorrectly enter this block.

Q4. *Survey completed*